

The AI

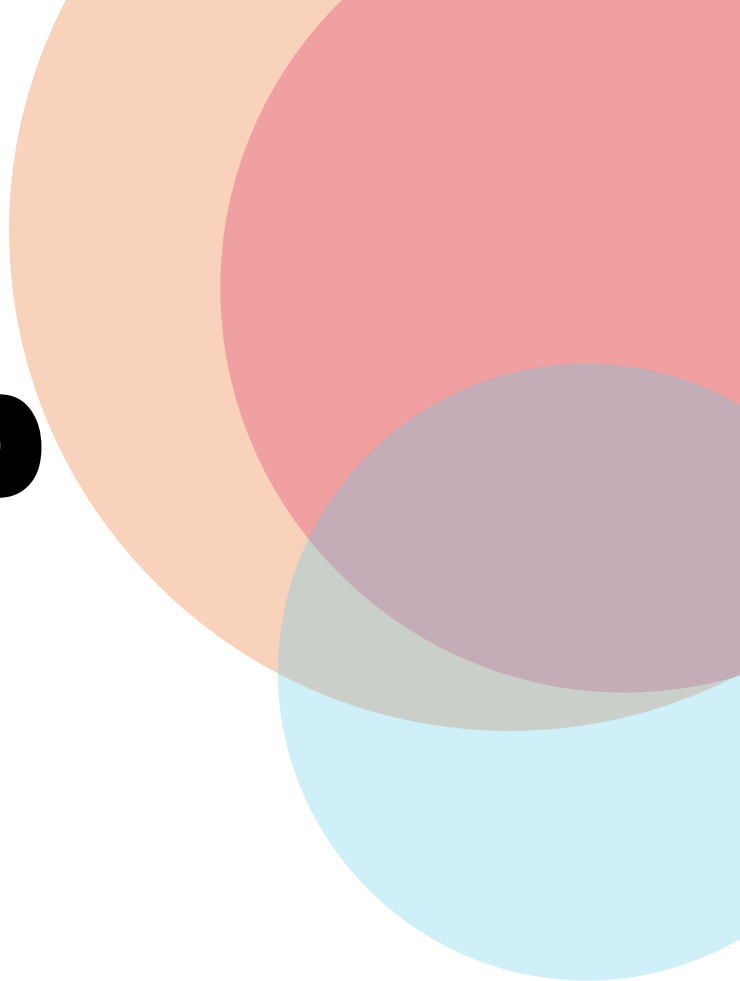
Fellowship

LLM Engineering Workshop

Prompt Engineering

Theory & Concepts — Session 2, Part A

Aryan | Robotics Lead @Hexaware



Agenda

01 Prompt Anatomy & Structure

Five components · Provider-specific ordering · Delimiters (XML vs Markdown) · Token budget allocation

02 Zero-Shot to Chain-of-Thought

Technique progression from zero-shot to CoT variants (Self-Consistency, ToT, GoT) and when to use each

03 System vs User Prompts

Instruction hierarchy across providers · What belongs where · Prompt injection defence

04 Structured Output Techniques

JSON mode vs constrained decoding vs Instructor library · The reasoning-then-format tradeoff

05 Failure Modes & Debugging

Four failure types · The ablation debugging method · Mitigating prompt drift

01

PROMPT ANATOMY & STRUCTURE

Five components

Provider ordering

Delimiters

Token budget

Three Numbers Every Prompt Engineer Should Know

>30%

The "Lost in the Middle" Penalty

Performance degradation when critical instructions are positioned in the **middle** of a long context. LLMs follow a U-shaped attention curve — strongest at the beginning and end.

Rule: Never bury your most critical instructions.

— Liu et al., "Lost in the Middle" (TACL 2024)

75%

Example Ordering Variance

Maximum performance fluctuation from **example ordering alone** in few-shot prompts — same examples, different sequence, near-chance to near-SOTA on GPT-3.

Rule: Place your most representative example last. Always test multiple orderings.

— Pezeshkpour et al. (NAACL 2024)

2–5×

Output Token Cost Multiplier

Output tokens cost **2–5× more** than input tokens on OpenAI and Anthropic. Optimising for concise outputs is often more cost-effective than trimming your prompt.

Rule: Always reserve 15–25% of your context window for output.

— OpenAI / Anthropic pricing documentation

The Five Components of a Well-Structured Prompt

1

Instruction

Action verb + task — what the model must do.

```
"Classify the following support ticket by urgency"
```

2

Context

Background the model needs to act correctly.

```
"Our company sells B2B SaaS. Urgency levels are P1-P4."
```

3

Examples

Input/output demonstrations (few-shot).

```
"Input: 'Server down' → Output: P1"
```

4

Constraints

Boundaries and exclusion rules.

```
"Respond in one word. Never explain your reasoning."
```

5

Output Format

Exact structure of the response.

```
Return JSON: {"urgency": "P1"|"P2"|"P3"|"P4"}
```

Provider Ordering Note

Components are universal — but the order you arrange them is model-specific.

OpenAI

Instructions first — or sandwich (start + end) for long prompts.
Use Markdown headers.

Anthropic

Instructions first, then context. XML tags are the native delimiter — Claude was trained on them.

Google Gemini

Context first, instructions at the very end. Supply all background before the directive.

Meta Llama

System prompt is the single most critical part. Few-shot examples go in the user turn.

→ See Slide 7 for full provider ordering breakdown

Advanced Chain-of-Thought Variants

Beyond standard CoT — when to go further

① Self-Consistency

Wang et al., ICLR 2023

HOW IT WORKS

Sample N reasoning paths independently, then majority-vote the final answer.

KEY RESULT

+17.9% over standard CoT on GSM8K
(74.4% vs 56.5%)

USE WHEN

Answer must be correct and latency is acceptable.

💰 5–40× base token cost

② Tree of Thoughts

Yao et al., NeurIPS 2023

HOW IT WORKS

BFS/DFS search over multiple reasoning branches with self-evaluation at each node.

KEY RESULT

Game of 24: 4% (CoT) → 74% (ToT)

USE WHEN

Problems require backtracking or exploratory planning.

💰 High — multiple model calls per branch

③ Graph of Thoughts

Besta et al., AAAI 2024

HOW IT WORKS

DAG-based reasoning — thoughts can be combined and refined, not just extended linearly.

KEY RESULT

62% lower error than ToT on sorting; 31% cheaper

USE WHEN

Problems with interdependent sub-problems.

💰 Moderate — more efficient than ToT

Note: For o1/o3 and DeepSeek-R1, explicit CoT adds negligible benefit — they reason internally via RL. Use zero-shot.

Provider-Specific Prompt Ordering

Same components — model-specific arrangement

O

OpenAI

Markdown headers (# ##)

RECOMMENDED ORDER

- 1 Role & Objective
- 2 Instructions
- 3 Reasoning Steps
- 4 Output Format
- 5 Examples
- 6 Context

🗨️ Sandwich critical instructions — place them at BOTH the beginning AND end for long prompts.

A

Anthropic

XML tags <tag>content</tag>

RECOMMENDED ORDER

- 1 Role (system prompt)
- 2 Instructions
- 3 Context
- 4 Examples
- 5 Output Format
- 6 Task (human turn)

🗨️ XML tags are a genuine architectural training choice for Claude — not just a style preference.

G

Google Gemini

Direct instructions + XML tags

RECOMMENDED ORDER

- 1 Context / background
- 2 Examples
- 3 Additional context
- 4 Instructions at the END

🗨️ Supply all context first. Place specific instructions at the very end — Gemini reads bottom-up.

L

Meta Llama

Chat template special tokens

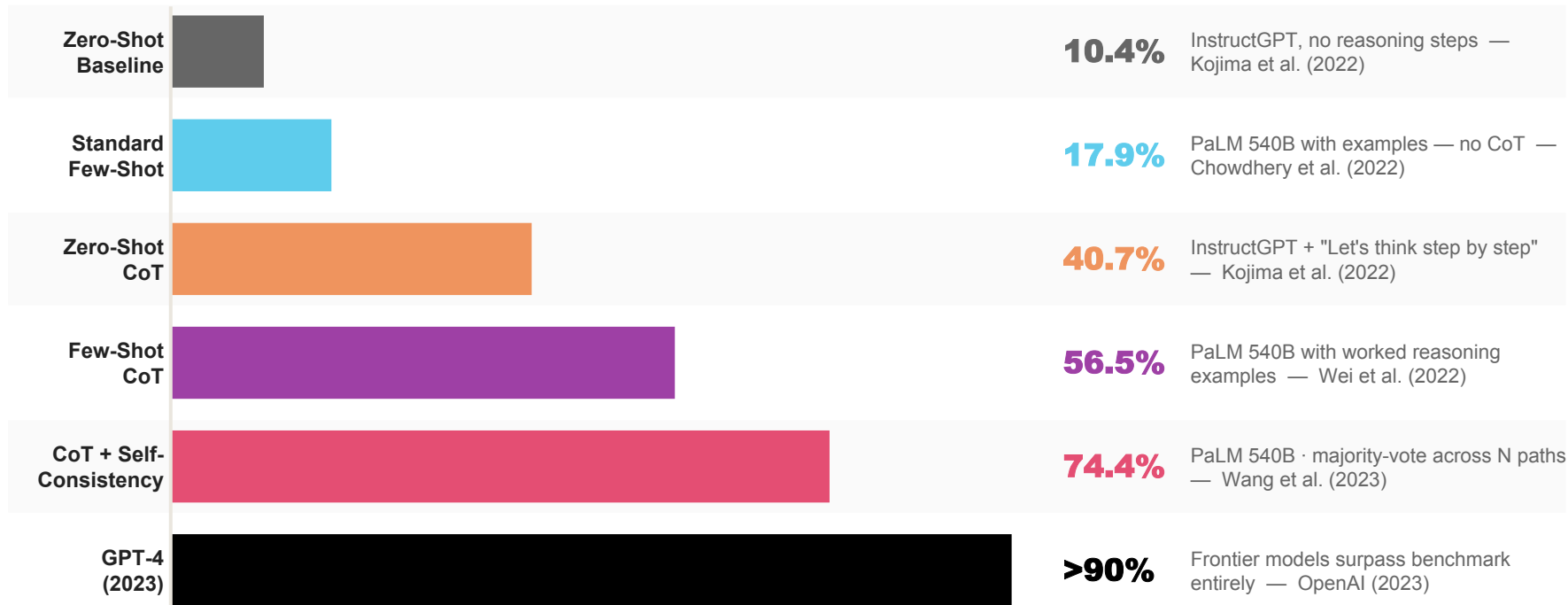
RECOMMENDED ORDER

- 1 System prompt ← CRITICAL
- 2 Few-shot examples
- 3 User instruction
- 4 Input data

🗨️ The system prompt is the single most important part of a Llama 4 prompt. Use chat template — don't skip it.

GSM8K Benchmark — Accuracy Progression

Grade-school math word problems · Each step = a prompting breakthrough



💡 Key insight: Each prompting technique stacks — zero-shot CoT alone gives 4× improvement; adding Self-Consistency pushes to 74.4%.

Prompting Techniques at a Glance

Choose based on task complexity, available examples, and latency tolerance

Technique	Best For	Token Cost	Accuracy Gain	Latency
Zero-Shot	Simple classification, factual Q&A, translation, reformatting	Lowest	Baseline	Fastest
Few-Shot (2–5 examples)	Format/style matching, domain-specific classification, non-obvious output formats	Medium	+10–30% on ambiguous tasks	Moderate
Zero-Shot CoT	Math/logic when no good examples exist. MultiArith: 17.7% → 78.7%	Medium	+30–70% on reasoning	Moderate
Few-Shot CoT	Complex multi-step reasoning where accuracy is critical. Outperforms zero-shot CoT consistently	Highest	+40–80% on reasoning	Slowest
Self-Consistency	When the answer must be correct. Majority-votes across N independent reasoning paths	5–40× base	+5–18% over CoT	Very slow

⚠ For o1 / o3 and DeepSeek-R1 (reasoning models): explicit CoT adds negligible benefit — they reason internally. Use zero-shot.



The single most impactful change you can make to any prompt is increasing specificity.

The Seven Dimensions of Specificity

Task

Scope

Audience

Format

Tone

Constraints

Quality Criteria

Vague vs Precise Instructions

The single most impactful change you can make to any prompt

× Vague Instruction

Summarize this document.

What's missing:

- × No length specified — model defaults to verbose
- × No focus area — output is generic and unfocused
- × No audience — wrong tone, wrong jargon level
- × No format — could be prose, bullets, or table

VS

✓ Precise Instruction

Summarize this document in 3 bullet points, each under 25 words.

Focus on technical architecture decisions and their rationale.

Target audience: engineering managers with no ML background.

Dimensions specified:

- Scope** 3 bullets · max 25 words each
- Focus** Architecture decisions + rationale
- Audience** Engineering managers, no ML background

🔗 Apply across all 7 dimensions: Task · Scope · Audience · Format · Tone · Constraints · Quality Criteria

Red Flags — Prompt Anti-Patterns

Recognise these patterns before your prompt reaches production

Vague Instructions

✗ "Make it better"



✓ "Improve readability by shortening sentences to under 20 words"

Conflicting Constraints

✗ "Be concise AND comprehensive"



✓ Choose one — or bound both: "Comprehensive but under 200 words"

Instruction Overload

✗ 15+ rules with exceptions



✓ Break into chained prompts. Keep top 5 rules. Switch to a reasoning model.

⚠ *Real case: Claude Haiku 3.5 hit a 49% error rate on a 15-rule document task — adding more rules made it worse.*

Double Negatives

✗ "Don't avoid mentioning..."



✓ "Make sure to mention..."

Debugging method — the Ablation Approach:

Remove one instruction at a time → test → when removal fixes the issue, rewrite that section → re-test full prompt.

AI for every human.

Session 2, Part A — Prompt Engineering: Theory & Concepts

www.theaifellowship.com

